| | | |
|---|---|---|
| TRƯỜNG: **ĐH CT TP.HCM**<br>KHOA: **CÔNG NGHỆ THÔNG TIN**<br>BỘ MÔN: **KHDL**<br>MH: **TH HỌC MÁY** | **BUỔI 1. HỒI QUY (REGRESSION)** | |

## A. MỤC TIÊU

- Xác định được bài toán hồi quy dựa trên dữ liệu thu thập được.
- Sử dụng thuật toán hồi quy phù hợp với bộ dữ liệu cụ thể.
- Đánh giá thực trạng bộ dữ liệu.
- Tiền xử lý dữ liệu (nếu có).
- Cài đặt thuật toán huấn luyện mô hình hồi quy cho bộ dữ liệu cụ thể: chia dữ liệu, chọn các siêu tham số,…).
- Đánh giá mô hình thông qua các độ đo.

## B. DỤNG CỤ - THIẾT BỊ THÍ NGHIỆM CHO MỘT SV

| STT | Chủng loại – Quy cách vật tư | Số lượng | Đơn vị | Ghi chú |
|---|---|---|---|---|
| 1 | Computer | 1 | 1 | |

## C. NỘI DUNG THỰC HÀNH

### I. Tóm tắt lý thuyết

**1. Sử dụng Google Colab để viết chương trình:**

B1. Truy cập địa chỉ: https://colab.research.google.com/

B2. Vào file → new notebook




B3. Kết nối máy chủ bằng cách click chuột vào Connect như trên hình.

**2. Linear Regression**

Hồi quy tuyến tính (Linear Regression) là một phương pháp phổ biến trong học máy để dự đoán giá trị của một biến phụ thuộc (biến đầu ra) dựa trên một hoặc nhiều biến độc

lập (biến đầu vào). Phương pháp này cố gắng tìm một đường thẳng tốt nhất (trong trường hợp hồi quy đơn) hoặc một mặt phẳng trong không gian đa chiều (trong trường hợp hồi quy đa biến) để mô hình hóa mối quan hệ giữa các biến.

Ví dụ: dữ liệu ngẫu nhiên (hoặc dữ liệu thực tế) để dự đoán mối quan hệ giữa hai biến.

```
# Thư viện cần thiết
    import numpy as np
    import matplotlib.pyplot as plt
    from sklearn.linear_model import LinearRegression
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import mean_squared_error, r2_score
# Tạo dữ liệu ngẫu nhiên
# x là biến đầu vào, y là biến đầu ra
    np.random.seed(0)
    x = 2 * np.random.rand(100, 1)
    y = 4 + 3 * x + np.random.randn(100, 1)  # Đường thẳng với nhiễu
# Chia dữ liệu thành tập huấn luyện và tập kiểm tra
    x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
random_state=42)
# Tạo mô hình hồi quy tuyến tính
    model = LinearRegression()
# Huấn luyện mô hình trên tập huấn luyện
    model.fit(x_train, y_train)
# Dự đoán trên tập kiểm tra
    y_pred = model.predict(x_test)
# Đánh giá mô hình
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
# In kết quả
    print(f"Hệ số chặn (Intercept): {model.intercept_}")
    print(f"Hệ số góc (Slope): {model.coef_}")
    print(f"Mean Squared Error (MSE): {mse}")
    print(f"R^2 score: {r2}")
```

```
# Vẽ biểu đồ
    plt.scatter(x_test, y_test, color="blue", label="Dữ liệu thực")
    plt.plot(x_test, y_pred, color="red", label="Dự đoán hồi quy")
    plt.title("Hồi quy tuyến tính đơn giản")
    plt.xlabel("Biến độc lập (x)")
    plt.ylabel("Biến phụ thuộc (y)")
    plt.legend()
    plt.show()
```

Kết quả:

Chương trình sẽ in ra các thông số của mô hình và biểu đồ minh họa hồi quy tuyến tính đơn giản.

Ví dụ:

Hệ số chặn (Intercept): [3.98105138]

Hệ số góc (Slope): [[2.91679257]]

Mean Squared Error (MSE): 0.8836872273331477

R^2 score: 0.9367661043361742

*** Hồi quy tuyến tính đa biến**

Nếu có nhiều hơn một biến đầu vào, có thể dễ dàng mở rộng hồi quy tuyến tính cho nhiều biến.

Ví dụ: với dữ liệu có nhiều biến đầu vào:

```
# Thư viện
    import numpy as np
    from sklearn.linear_model import LinearRegression
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import mean_squared_error
# Tạo dữ liệu giả lập với nhiều biến đầu vào
    np.random.seed(0)
    X = 2 * np.random.rand(100, 3)  # 3 biến đầu vào
    y = 4 + 3 * X[:, 0] + 2 * X[:, 1] + 5 * X[:, 2] + np.random.randn(100, 1)
# Chia dữ liệu
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
# Tạo mô hình hồi quy tuyến tính
    model = LinearRegression()
# Huấn luyện mô hình
    model.fit(X_train, y_train)
# Dự đoán trên tập kiểm tra
    y_pred = model.predict(X_test)
# Đánh giá mô hình
    mse = mean_squared_error(y_test, y_pred)
# In kết quả
    print(f"Hệ số chặn (Intercept): {model.intercept_}")
    print(f"Hệ số hồi quy (Coefficients): {model.coef_}")
    print(f"Mean Squared Error (MSE): {mse}")
```

**Giải thích**:

- Với hồi quy đa biến, X là ma trận với nhiều cột (mỗi cột là một biến độc lập).
- Kết quả sẽ cho hệ số hồi quy (slope) cho từng biến đầu vào.

### 3. Logistic Regression

Một thuật toán phân loại phổ biến, được sử dụng để dự đoán khả năng xảy ra của một biến nhị phân (phân loại nhị phân, ví dụ như 0 hoặc 1, có hoặc không). Thay vì dự đoán một giá trị liên tục như trong hồi quy tuyến tính, hồi quy logistic dự đoán xác suất mà một mẫu thuộc về một lớp nhất định, và sau đó áp dụng một ngưỡng để phân loại nó. Công thức hồi quy logistic sử dụng hàm sigmoid để chuyển đổi đầu ra của mô hình thành một xác suất (giá trị từ 0 đến 1):

$$\hat{y} = \frac{1}{1 + e^{-(\theta_0 + \theta_1 \cdot x_1 + \theta_2 \cdot x_2 + \cdots + \theta_n \cdot x_n)}}$$

Trong đó:

- $\hat{y}$ là xác suất mẫu thuộc về lớp 1.
- $x_1, x_2, \ldots, x_n$ là các biến đầu vào.
- $\theta_0, \theta_1, \ldots, \theta_n$ là các tham số (hệ số) của mô hình.

**Hồi quy Logistic với bộ dữ liệu hoa Iris:**

```
# Thư viện
    import numpy as np
```

```
        import matplotlib.pyplot as plt
        from sklearn.datasets import load_iris
        from sklearn.linear_model import LogisticRegression
        from sklearn.model_selection import train_test_split
        from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
# Tải dữ liệu hoa Iris trong thư viện sklearn
        iris = load_iris()
        X = iris['data']  # Các đặc trưng (features)
        y = iris['target']  # Các nhãn (target)
# Chọn ra hai loại hoa (0 và 1) để thực hiện phân loại nhị phân
        X = X[y != 2]  # Loại bỏ lớp 2
        y = y[y != 2]  # Chỉ giữ lại lớp 0 và 1
# Chia dữ liệu (minh họa chia 80% huấn luyên và 20% kiểm tra)
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# Tạo mô hình Logistic Regression
        model = LogisticRegression()
# Huấn luyện mô hình trên tập huấn luyện
        model.fit(X_train, y_train)
# Dự đoán trên tập kiểm tra
        y_pred = model.predict(X_test)
# Đánh giá mô hình
        accuracy = accuracy_score(y_test, y_pred)
        conf_matrix = confusion_matrix(y_test, y_pred)
        class_report = classification_report(y_test, y_pred)
# In kết quả
        print(f"Độ chính xác (Accuracy): {accuracy}")
        print("Ma trận nhầm lẫn (Confusion Matrix):")
        print(conf_matrix)
        print("Báo cáo phân loại (Classification Report):")
        print(class_report)
```

## III. Bài tập ở lớp

Bài 1. Giả sử có một tập dữ liệu giả định với các cột Diện tích nhà (m2) và Giá bán (triệu VND).

1. Cần xây dựng một mô hình hồi quy tuyến tính đơn giản để dự đoán Giá bán dựa trên Diện tích nhà.
2. Thêm nhiều (số lượng) dữ liệu và độ nhiễu để xem mô hình hoạt động như thế nào.
3. Thêm các biến đầu vào khác để nâng cao tính phức tạp của mô hình.

Bài 2. Dự đoán khả năng mắc bệnh dựa trên chỉ số BMI

Giả sử có một tập dữ liệu với các thông tin sau:

- BMI: Chỉ số khối cơ thể của bệnh nhân (Body Mass Index).

- Mắc bệnh: Nhãn mục tiêu là 1 nếu bệnh nhân mắc bệnh và 0 nếu không mắc bệnh.

Yêu cầu:

1. - Tạo dữ liệu mẫu giả lập với đặc trưng BMI và nhãn Mắc bệnh dựa trên mối liên hệ logic (BMI cao có thể dẫn đến khả năng mắc bệnh cao hơn).

2. - Sử dụng thư viện phù hợp (HD: scikit-learn) để xây dựng một mô hình hồi quy logistic.

3. - Huấn luyện mô hình trên tập dữ liệu mẫu.

4. - Dự đoán xác suất và nhãn mắc bệnh cho một tập dữ liệu kiểm tra.

5. - Đánh giá hiệu quả của mô hình thông qua độ chính xác và ma trận nhầm lẫn.

6. - Trực quan hóa dữ liệu và đường hồi quy logistic.

Bài 3. Hãy chọn 1 bộ dữ liệu tùy ý (HD: có thể tham khảo trên các trang mạng, ví dụ như Kaggle) và thực hiện các bước cần thiết để có được mô hình hồi qui ứng với bộ dữ liệu đã chọn.

## IV. Bài tập về nhà

Bài 1. Thay đổi điều kiện sinh dữ liệu, ví dụ tăng độ nhiễu để kiểm tra độ chính xác của mô hình.

Bài 2. Đổi ngưỡng dự đoán xác suất (ví dụ 0.6, 0.7,… thay vì 0.5) và quan sát sự thay đổi trong kết quả dự đoán.

Bài 3. Hồi quy tuyến tính với tập dữ liệu giá nhà ở

Mô tả: Sử dụng thuật toán Linear Regression để dự đoán giá nhà dựa trên một số thuộc tính.

Yêu cầu:

- Tải tập dữ liệu Boston Housing từ sklearn.datasets (có thể tải từ kaggle).
- Phân chia dữ liệu thành tập huấn luyện và tập kiểm tra.
- Huấn luyện mô hình hồi quy tuyến tính trên tập huấn luyện và dự đoán trên tập kiểm tra.
- Tính toán các chỉ số đánh giá: MSE (Mean Squared Error) và $R^2$ score cho tập kiểm tra.